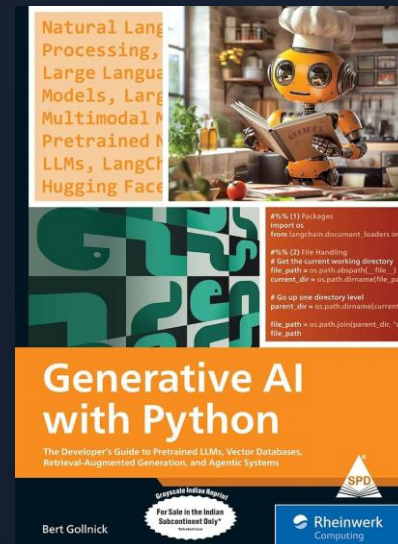


CHAPTER 2

PRETRAINED MODELS

Hugging Face • NLP Tasks • Generative Models



Chapter 2 — Learning Roadmap

Conceptual understanding of pretrained, task-specific models

Pretrained Models

Why reuse an existing model?

Text Summarization

Condense long text

Zero-Shot Classification

Classify without task-specific training

Question Answering

Answer from context

Text-to-Image

Generate images from prompts

Hugging Face

Find models, datasets and tools

Translation

Convert between languages

Fill-Mask

Predict missing tokens

NER

Extract named entities

Text-to-Audio

Generate music from prompts

Section-Wise Class Distribution (2.1 - 2.9)

Class 1: Ecosystem & Sequence Tasks

- 2.1 Hugging Face Platform:**
- 2.2 Text Summarization:**
- 2.3 Translation**

Class 2: Contextual NLP Mechanics

- 2.4 Zero-Shot Classification:** NLI premise-hypothesis entailment scoring.
- 2.5 Fill-Mask Task:** Masked Language Modeling (MLM) & BERT.
- 2.6 Question Answering:** Extractive span pointer detection & SQuAD 2.0.

Class 3: Token Tagging & Alignment

- 2.7 Named Entity Recognition (NER):** BIO tagging scheme & subword alignment.
- 2.7 Anonymization Workflows:** Entity boundary detection & privacy masking.

Class 4: Multimodal Synthesis & Tradeoffs

- 2.8 Text-to-Image:** CLIP embeddings, Latent U-Net denoising & VAE.
- 2.9 Text-to-Audio:** MusicGen, EnCodec residual codebooks & WAV export.

Chapter 2 — Learning Roadmap

Conceptual understanding of pretrained, task-specific models

Pretrained Models

Why reuse an existing model?

Text Summarization

Condense long text

Zero-Shot Classification

Classify without task-specific training

Question Answering

Answer from context

Text-to-Image

Generate images from prompts

Hugging Face

Find models, datasets and tools

Translation

Convert between languages

Fill-Mask

Predict missing tokens

NER

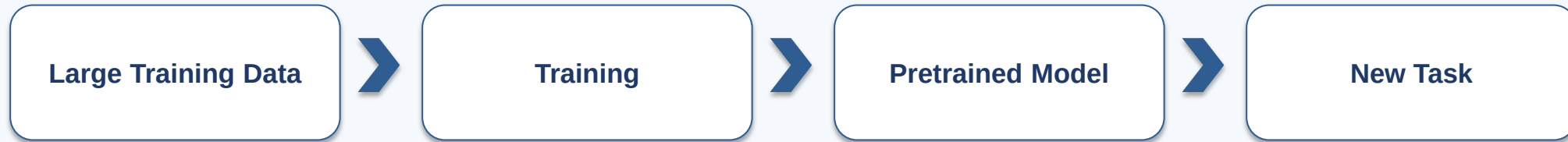
Extract named entities

Text-to-Audio

Generate music from prompts

What is a Pretrained Model?

A model that has already learned from data and can be reused



- Training is performed before the model is used for our specific application.
- The learned parameters contain patterns and representations acquired during previous training.
- The model may be used directly for a supported task or adapted/fine-tuned for a new dataset.
- Pretrained, task-specific models can be much smaller and easier to run locally than very large models.

Core idea

Reuse learned capability instead of rebuilding everything from scratch.

Classic ML Models vs. Large Language Models

Why begin with task-specific pretrained models?

Classic / Task-Specific ML

- One model is commonly built for one task
- Often trained with labeled data
- Narrow, specialized capability
- Usually smaller
- Often practical to run locally

Pretrained Specialized Model

- Already trained for a particular task
- Can often be used out of the box
- Reusable through a model interface
- Smaller than a general-purpose LLM
- Useful when the task is well defined

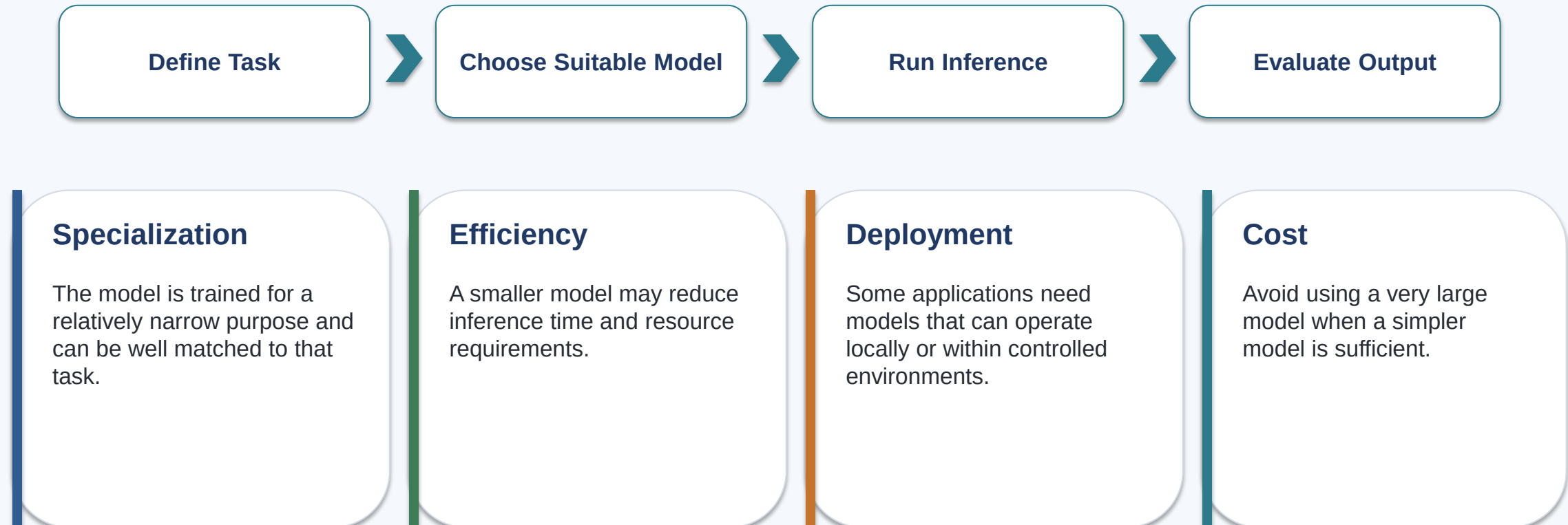
LLM / Large Model

- Trained on very large and often diverse datasets
- Can support many language capabilities
- Powerful and flexible
- Higher inference cost may occur
- Local deployment can be difficult for large models

Practical principle: use a simple model when the task is simple; use a larger model when the task requires it.

Why Use a Task-Specific Pretrained Model?

Model choice is a balance between capability, cost, latency and deployment



2.1 Hugging Face

An ecosystem for sharing and using machine-learning models and datasets

HUGGING FACE

- AI company and open-source platform/ecosystem.
- Provides access to pretrained models for NLP, computer vision and other ML tasks.
- Provides datasets, libraries, model demonstrations and community resources.
- Enables developers to discover, evaluate and reuse models rather than building every model from scratch.

Models

Datasets

Transformers

Tokenizers

Diffusers

Spaces

Hugging Face: What Can You Find?

The platform covers more than language models

Natural Language

- Summarization
- Translation
- Classification
- Question answering
- Named entity recognition
- Fill-mask

Computer Vision

- Image tasks
- Image generation
- Vision models
- Multimodal applications

Audio

- Speech-related tasks
- Audio generation
- Music generation
- Audio processing

Tools & Community

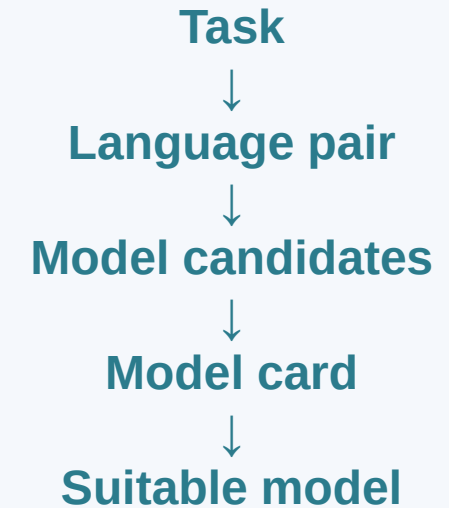
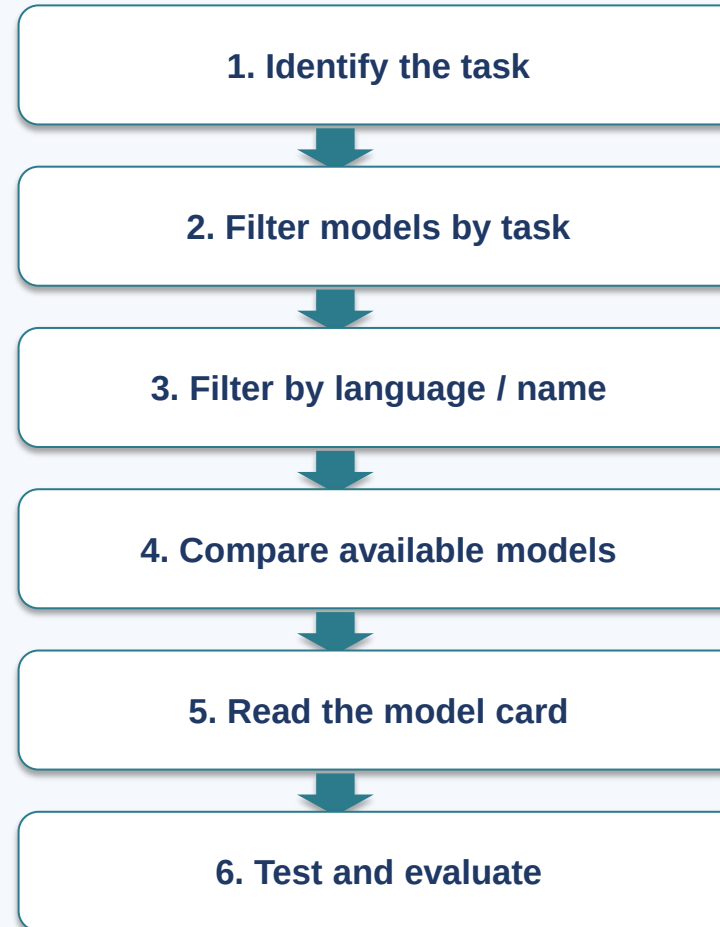
- Model cards
- Datasets
- Libraries
- Spaces / demos
- Community information

Finding the Right Model

Model selection starts with the task

Example: Translation

- Select Translation as the task.
- Use source/target language information to narrow the results.
- Compare models using available information such as popularity, downloads or other indicators.
- Read the model card before adopting the model.



Hugging Face Model Card

The model card is the first place to understand a pretrained model

Purpose

What the model does and what task it is intended for.

Evidence

Training information, evaluation metrics, benchmarks and performance information.

Practical Use

Usage instructions, files/model size, limitations, supported languages and license.

Student habit: never select a model only by its name—read its model card.

2.2 Text Summarization

Generating a shorter version of a longer text while preserving important information

Long Input

A long research article or document contains more information than the reader may need at once.

The task is to identify and express the important information in a shorter form.

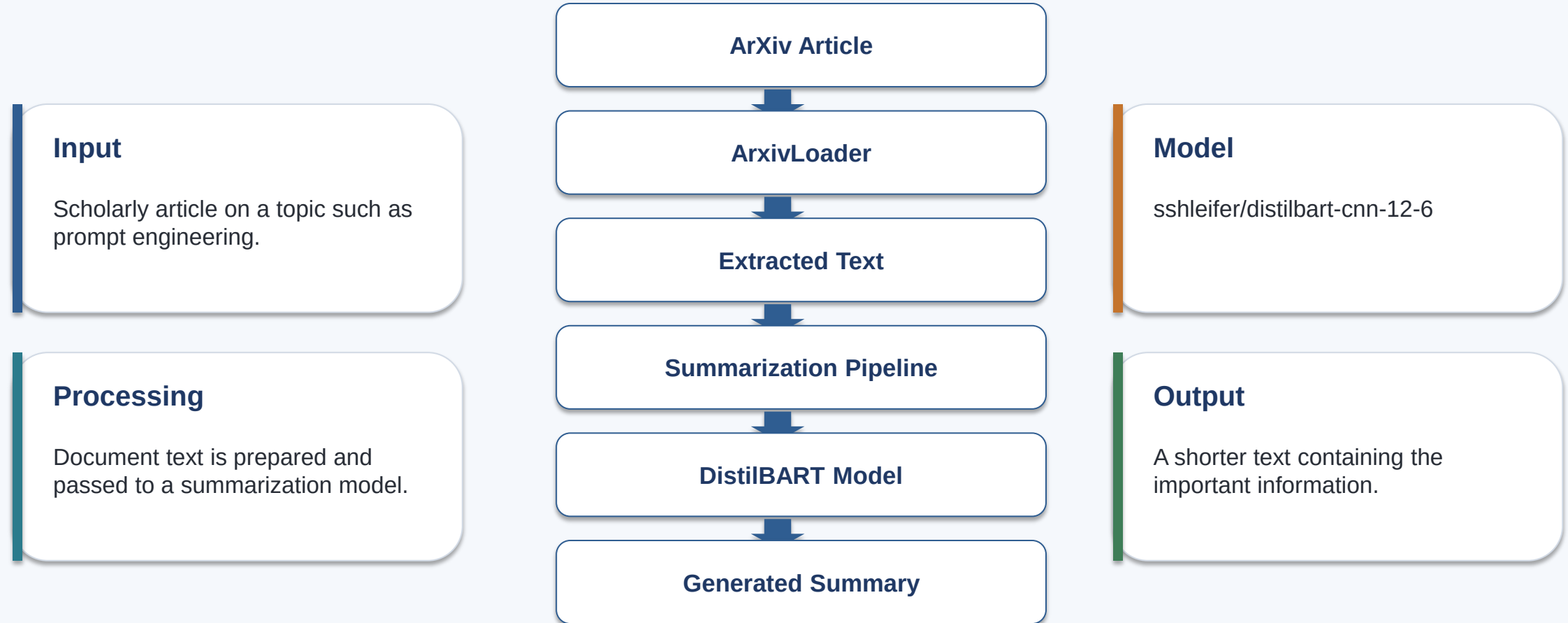
Shorter Output

A summary condenses the source while attempting to preserve its key ideas, facts and overall meaning.

Example: Research paper → key findings, methods, motivation and conclusions in a compact form.

Text Summarization: End-to-End Flow

The book demonstrates summarizing an arXiv article



What Happens Inside a Summarization Pipeline?

A conceptual view—without writing the implementation

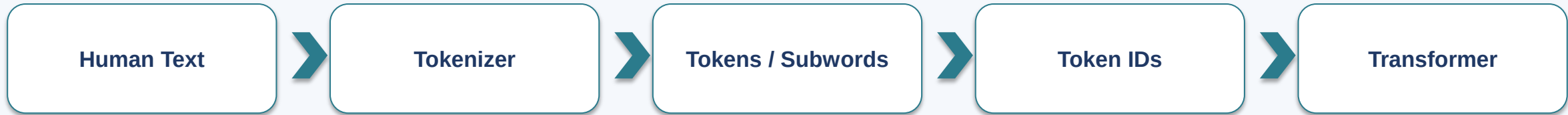


The tokenizer converts text into model-readable tokens/IDs. The Transformer processes the sequence and generates output tokens. The tokenizer/decode stage converts the generated representation back into readable text.

Key idea: the pipeline hides many preprocessing and postprocessing steps so the user can focus on the task.

Tokenizer: Why Do We Need It?

A tokenizer is the bridge between human language and model input



Token

A unit of text; it may be a word, subword or punctuation depending on the tokenizer.

Token ID

A numerical identifier from the tokenizer's vocabulary.

Model Input

The numerical representation is passed to the neural network.

Tokenizer Process

A simplified view of the stages before the Transformer

1. Original Text

2. Normalization

3. Pre-tokenization

4. Tokenization

5. Vocabulary Mapping

6. Special Tokens + Attention Mask

The exact behavior depends on the tokenizer associated with the pretrained model.

Transformer Models

The architecture behind many modern pretrained models

Core idea

- Transformers process sequences using attention mechanisms.
- They can learn relationships between tokens and their context.
- Different Transformer models are pretrained for different objectives and tasks.

Examples

BERT

Encoder-based language understanding

BART

Sequence-to-sequence generation

RoBERTa

Optimized BERT-style encoder

DistilBART

Smaller distilled BART model

Text Summarization Model: DistilBART

The model used in the book

Model

sshleifer/distilbart-cnn-12-6

A pretrained summarization model based on the BART family and distilled to a smaller model.

Why It Fits the Task

- Designed for summarization
- Can be used through a summarization pipeline
- More compact than a full-size model
- Suitable for a focused task

What Students Should Check

- Model card
- Input/output expectations
- Model size
- Performance information
- Deployment constraints

Task → model → pipeline → inference → summary

Text Summarization — Key Takeaways

- Summarization reduces a long document to a shorter representation of its important information.
- ArxivLoader can provide article text; a pretrained summarization model performs the generation.
- The Hugging Face pipeline connects the task with the selected pretrained model.
- Tokenization converts text into a representation that the Transformer can process.
- Model cards help us understand how a specific model should be used and what its limitations are.

Bridge to the next section

The same model-selection pattern can now be applied to other tasks.

2.3 Translation

Automatically converting text from one language into another



Task

Translate a sentence while preserving its meaning in the target language.

Model

A pretrained model is selected for the required language pair.

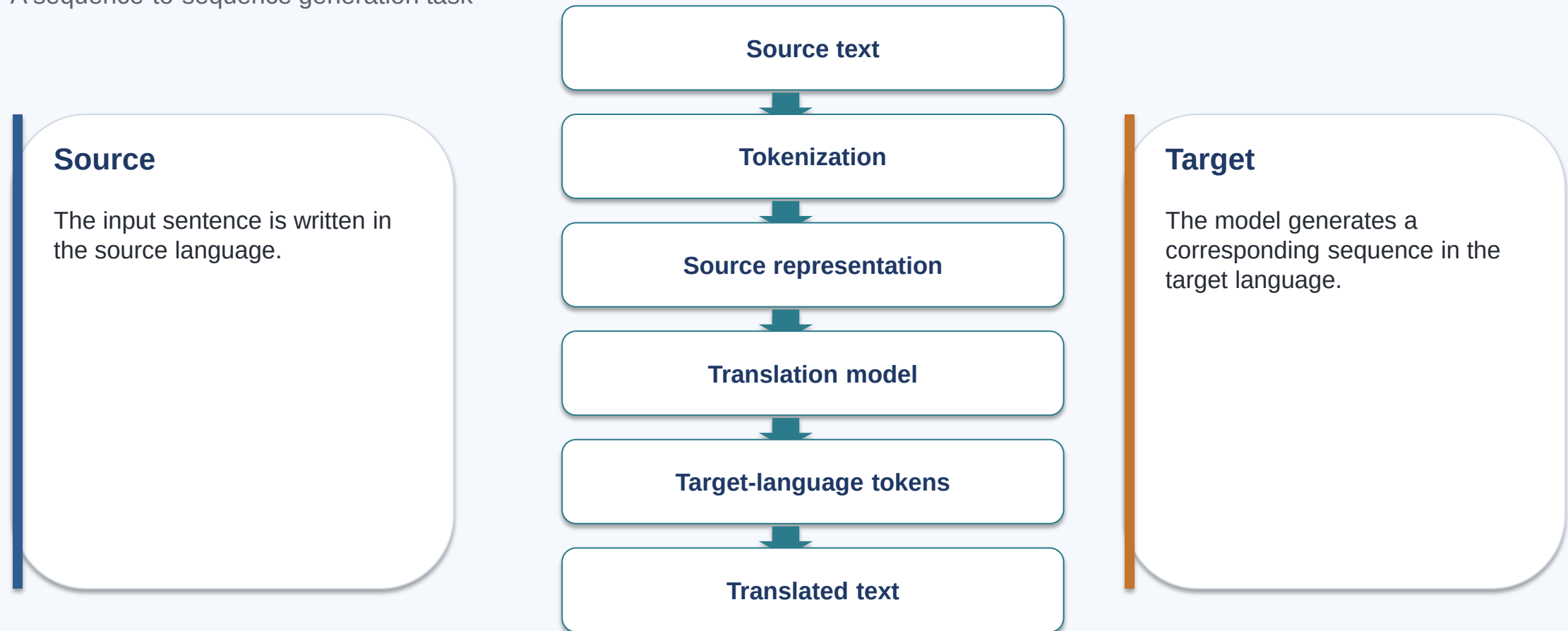
Output

Text expressed in the target language.

Book example: English → Japanese

Translation: How the Model Works

A sequence-to-sequence generation task



The model must learn relationships between source-language patterns and target-language expressions.

2.3.1 Translation — Task

The chapter asks students to solve a translation problem independently

Source Sentence

“Be the change you wish to see in the world.”

Language: English

Required Output

Translate the sentence into Japanese.

Goal: preserve the intended meaning while expressing it naturally in the target language.

The important skill is not memorizing a model—it is knowing how to find a suitable model for the task.

2.3.2 Translation — Model Selection

The book narrows the Hugging Face search by language pair



Selected Model

Mitsua/elan-mt-bt-en-ja

Language Pair

English → Japanese

Selection Lesson

Use the model name, task filters and model card to narrow a large model collection.

Translation — Key Takeaways

- Translation is a sequence-to-sequence language task.
- The required language pair is an important model-selection criterion.
- The book uses Mitsua/elan-mt-bt-en-ja for English-to-Japanese translation.
- Different translation models may support different languages and may have different quality and resource requirements.

General pattern: identify task → narrow model choices → read model card → test the model.

2.4 Zero-Shot Classification

Classifying text into candidate categories without training a new classifier for those categories

Traditional Classification

Training examples are labeled with known categories.

The classifier learns a mapping from input features to those predefined classes.

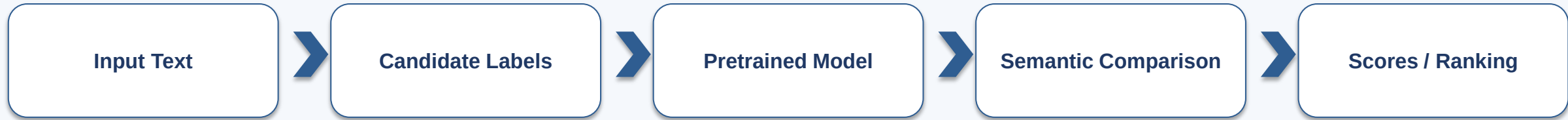
A new category normally requires additional training data and model adaptation.

Zero-Shot Classification

A pretrained model receives the input text and candidate labels at inference time.

It uses its learned language/semantic knowledge to estimate how well the text matches each label.

Zero-Shot Classification: Conceptual Flow



Input

A text passage whose category is unknown.

Labels

Categories such as romance, fantasy and crime are supplied by the user.

Output

The model assigns scores and ranks the candidate labels.

Zero-Shot Example

Genre classification from short book passages

Pride and Prejudice

Candidate label: romance

The model receives only a passage, not a task-specific training set for this classification exercise.

Alice's Adventures in Wonderland

Candidate label: fantasy

The model uses the passage's semantic content to compare it with the supplied labels.

The Return of Sherlock Holmes

Candidate label: crime

The book reports that the model assigns the crime label with a very high probability.

Zero-Shot Model: BART Large MNLI

The model selected in the chapter

Model

facebook/bart-large-mnli

A BART-based pretrained model fine-tuned for natural-language inference (MNLI).

Why It Fits

The model can compare a text input with candidate hypotheses/labels and produce scores useful for zero-shot classification.

Student takeaway

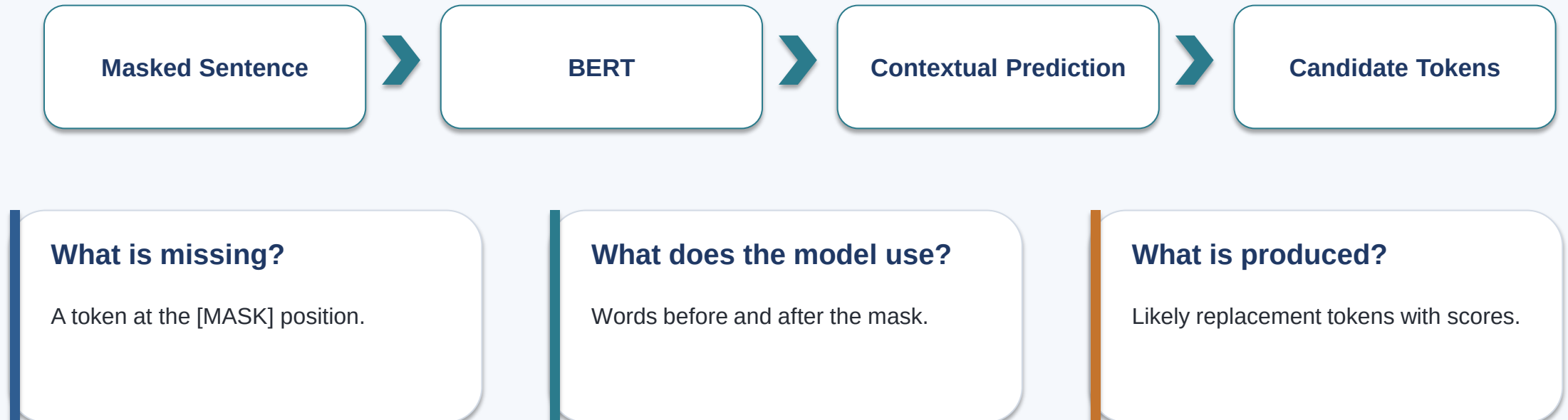
Zero-shot classification does not mean “no training ever.”

It means the model can classify new candidate categories without a separate task-specific classifier being trained for those categories.

2.5 Fill-Mask

Predicting a missing token from its surrounding context

Example: “AI is transforming the [MASK] industry.”



Fill-Mask: Why BERT Is Suitable

Masked Language Modeling

BERT

Bidirectional Encoder Representations from Transformers.

BERT was pretrained with masked-language-modeling objectives, making missing-token prediction a natural use case.

Sentence with a missing token



BERT reads the surrounding context



Predicts likely tokens



Ranks the predictions

Book Model

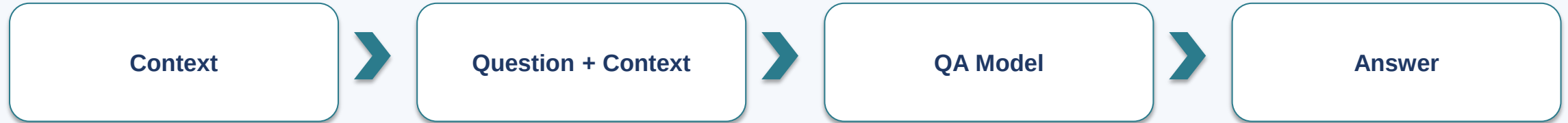
bert-base-uncased

The chapter uses this model for the fill-mask example.

The model can propose alternatives such as role, fashion, super or business for “I am a [MASK] model.”

2.6 Question Answering

Answering questions using information contained in a supplied context



Context

A passage containing relevant information.

Question

The information the user wants to retrieve.

Answer

The model locates an answer span in the supplied context.

Question Answering: Context Is Critical

The chapter uses a narrow, context-based QA model

Example Context

“Remote work allows employees to work from anywhere, providing flexibility and a better work-life balance. It reduces commuting time...”

Question → Answer

Question: What are the benefits of remote work?

Answer: flexibility and a better work-life balance

The model extracts the answer from the supplied context.

Important limitation: the model relies on the supplied context and does not use external knowledge.

Question Answering Model: RoBERTa SQuAD2

The model selected in the chapter

Model

deepset/roberta-base-squad2

A RoBERTa-based question-answering model trained for SQuAD 2.0.

SQuAD 2.0

The dataset includes questions that may be answerable or unanswerable from the supplied context.

This challenges QA systems to recognize when the context does not provide an answer.

Model Behavior

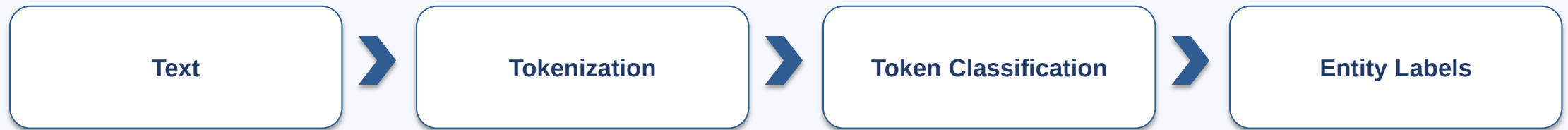
Question + context → answer span or inability to find a supported answer.

The chapter emphasizes context-based inference.

2.7 Named Entity Recognition (NER)

Identifying and categorizing important entities in text

“My name is Bert and I live in Hamburg.”



PERSON

Bert

LOCATION

Hamburg

Other possible types

Organization, date, quantity,
miscellaneous entities

NER: From Unstructured Text to Structured Information

Why NER is useful

Information Extraction

Automatically identify names, places, dates and other relevant entities.

Search & Analysis

Convert free text into structured signals that can be searched, counted or analyzed.

Anonymization

Identify persons, places and dates so sensitive entities can be removed or masked.

NER Model: BERT Token Classification

The model selected in the chapter

Model

dslim/bert-base-NER

A BERT-based model designed for named entity recognition.

Task Type

Token classification

The model assigns an entity label to tokens/token spans.

Example Output

Bert → B-PER

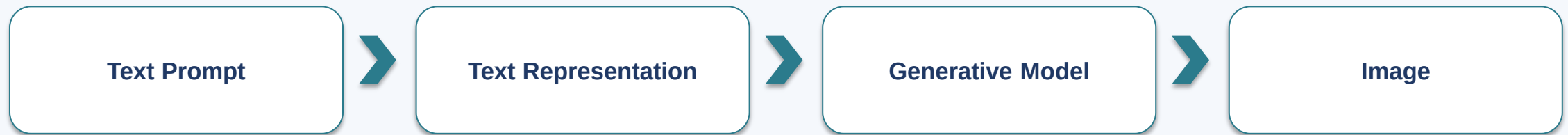
Hamburg → B-LOC

The chapter reports very high scores for these two detections in its example.

2.8 Text-to-Image

Generating an image from a textual prompt

Prompt: “dog”



Input

A textual description of the desired visual content.

Generation

A pretrained generative model produces visual content from the prompt.

Output

A newly generated image.

Text-to-Image: Model and Resources

The chapter deliberately chooses a relatively small model

Model

amused/amused-256

Produces 256×256 pixel images in the chapter example.

Technology

The example uses the Diffusers ecosystem for text-to-image generation.

Resource Awareness

Text-to-image models can be resource intensive.

The book notes that its CPU example took about half an hour, illustrating why model size and hardware matter.

Diffusion-Based Image Generation — High-Level View

Conceptual understanding rather than implementation details

Diffusers

A library/ecosystem used to work with diffusion-based generative models.

Text Prompt



Text / semantic representation



Generative process



Image representation



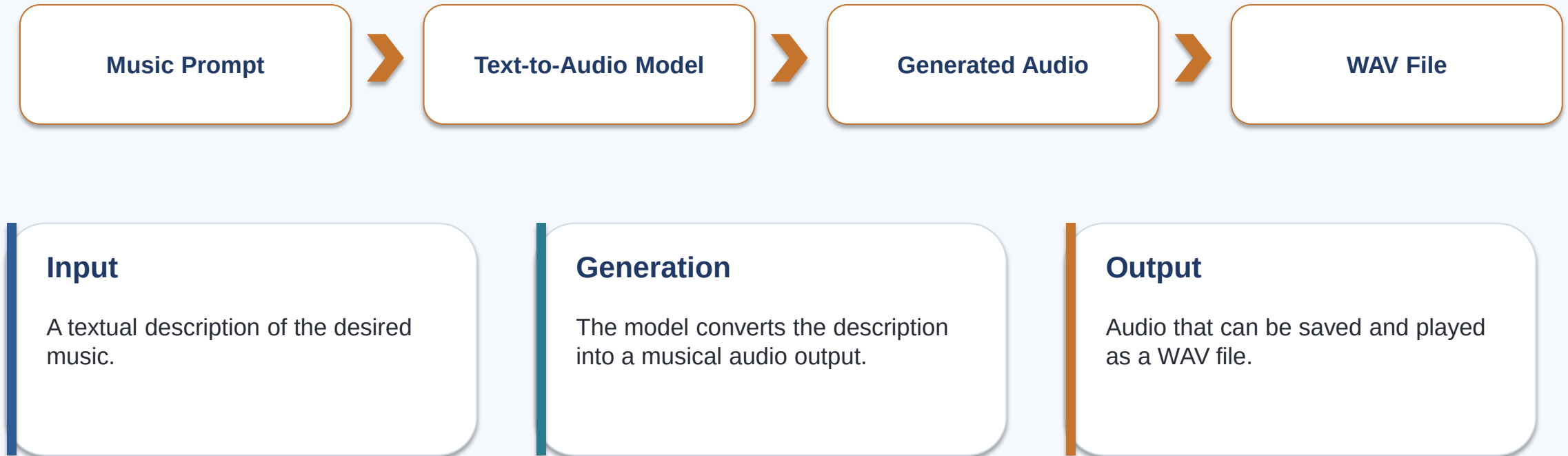
Generated image

Why the model card matters

The book notes that much of the implementation can be taken directly from the model card, making the documented model interface important.

2.9 Text-to-Audio

Generating audio—specifically music in the chapter exercise—from a text description



2.9.1 Text-to-Audio — Task

The chapter asks students to find a model and run it locally

Task

Find and select a pretrained model that can create a music track from a text description.

Then run the model locally.

Resource Consideration

If hardware is limited, search for a smaller model.

The chapter specifically suggests looking for models with “small” in their name.

The task reinforces the chapter's main skill: selecting a model based on the task and available resources.

2.9.2 Text-to-Audio — Model

facebook/musicgen-small

Selected Model

facebook/musicgen-small

A pretrained model used for text-conditioned music generation.

Prompt Example

“lo-fi music with a soothing melody”

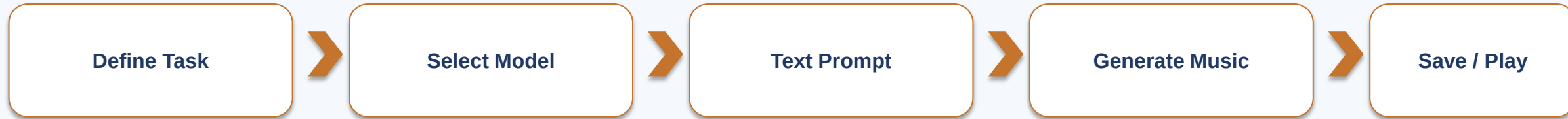
The text prompt describes the musical characteristics to generate.

Output

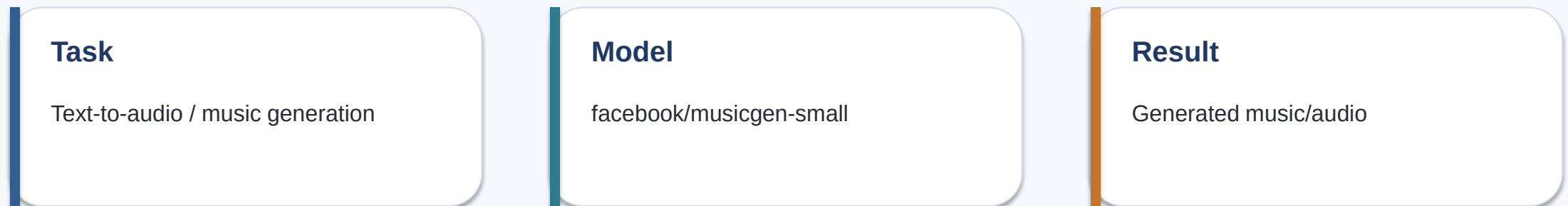
Generated audio is represented as audio data and saved as a WAV file in the chapter's example.

Text-to-Audio: Conceptual Pipeline

The same reusable pattern appears again



Task + model selection + prompt + inference + output handling



Comparing the Chapter's Pretrained Models

Different tasks require different model capabilities

Task	Input	Output	Model in the chapter
Summarization	Long text	Shorter text	sshleifer/distilbart-cnn-12-6
Translation	English text	Japanese text	Mitsua/elan-mt-bt-en-ja
Zero-shot	Text + labels	Label scores	facebook/bart-large-mnli
Fill-mask	Text + mask	Likely tokens	bert-base-uncased
Question answering	Question + context	Answer	deepset/roberta-base-squad2
NER	Text	Entities + labels	dslim/bert-base-NER
Text-to-image	Text prompt	Image	amused/amused-256
Text-to-audio	Music prompt	Audio	facebook/musicgen-small

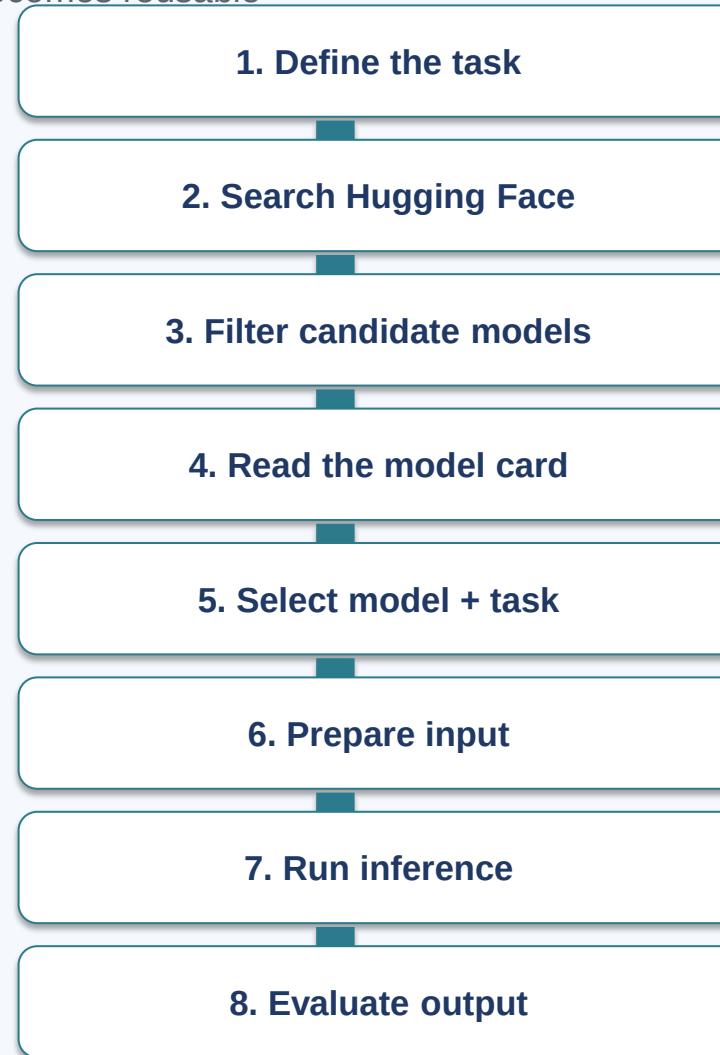
Model names are taken from the chapter examples; actual suitability depends on the intended application.

The Common Pattern Across Chapter 2

Once you understand one task, the workflow becomes reusable

The chapter is not mainly about memorizing APIs.

It teaches a reusable way to think about pretrained-model selection and application.



Task
→ Model
→ Input
→ Inference
→ Output

Then evaluate whether the result is suitable.

How to Select a Pretrained Model?

A practical checklist for students

1 Identify the task

Translation? QA? NER? Generation?

2 Check the input

Text, image, audio or another modality?

3 Define the output

Text, label, entity, image or audio?

4 Check capabilities

Language, task support and intended use

5 Check resources

Model size, CPU/GPU needs and latency

6 Read the model card

Training, evaluation, limitations, usage and license

Key takeaway: choose the simplest pretrained model that adequately meets the task requirements.

Chapter 2 — Final Summary

- Pretrained models allow us to reuse learned capabilities instead of training every model from scratch.
- Hugging Face provides a practical ecosystem for discovering models, datasets, tools and model documentation.
- Task-specific models can be preferable to very large models when the task is simple, latency matters, or local deployment is required.
- Different tasks require different pretrained models: summarization, translation, zero-shot classification, fill-mask, QA, NER, text-to-image and text-to-audio.
- Model cards are essential for understanding intended use, capabilities, limitations and deployment considerations.
- The recurring skill is: identify the task → select the model → understand the model → run inference → evaluate the result.

Chapter 2 — What Students Should Be Able to Explain

Concepts

- What a pretrained model is
- Why model selection matters
- What Hugging Face provides
- What a model card contains

Techniques

- Summarization
- Translation
- Zero-shot classification
- Fill-mask
- Question answering
- NER

Generative Tasks

- Text-to-image
- Text-to-audio
- Resource considerations
- Model/task matching
- Inference workflow

Capstone project intentionally omitted, as requested.